

APIからのデータを可視化する

APIからとったjsonを可視化

rkskmt

1

前回のあらすじ

2

大きな山を見つけた

- 前回： 記事「七夕」の 2025年まるごと1年分 の日別アクセス数を取得した
- 最大が中央値の約280倍という違和感から、最後に折れ線を描いた

Code

```
1 import requests
2 import numpy as np
3
4 url = ("https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article/"
5       "ja.wikipedia.org/all-access/all-agents/"
6       "七夕/daily/20250101/20251231")
7 headers = {
8     "User-Agent": "IP3200-lecture/1.0 (https://rkskmt.github.io/ip3200/)"
9 }
10
11 response = requests.get(url, headers=headers)
12 data = response.json()
13
14 views = np.array([item["views"] for item in data["items"]])
```

統計量	値
最大	52,168回
最小	88回
平均	539.1回
中央値	187回

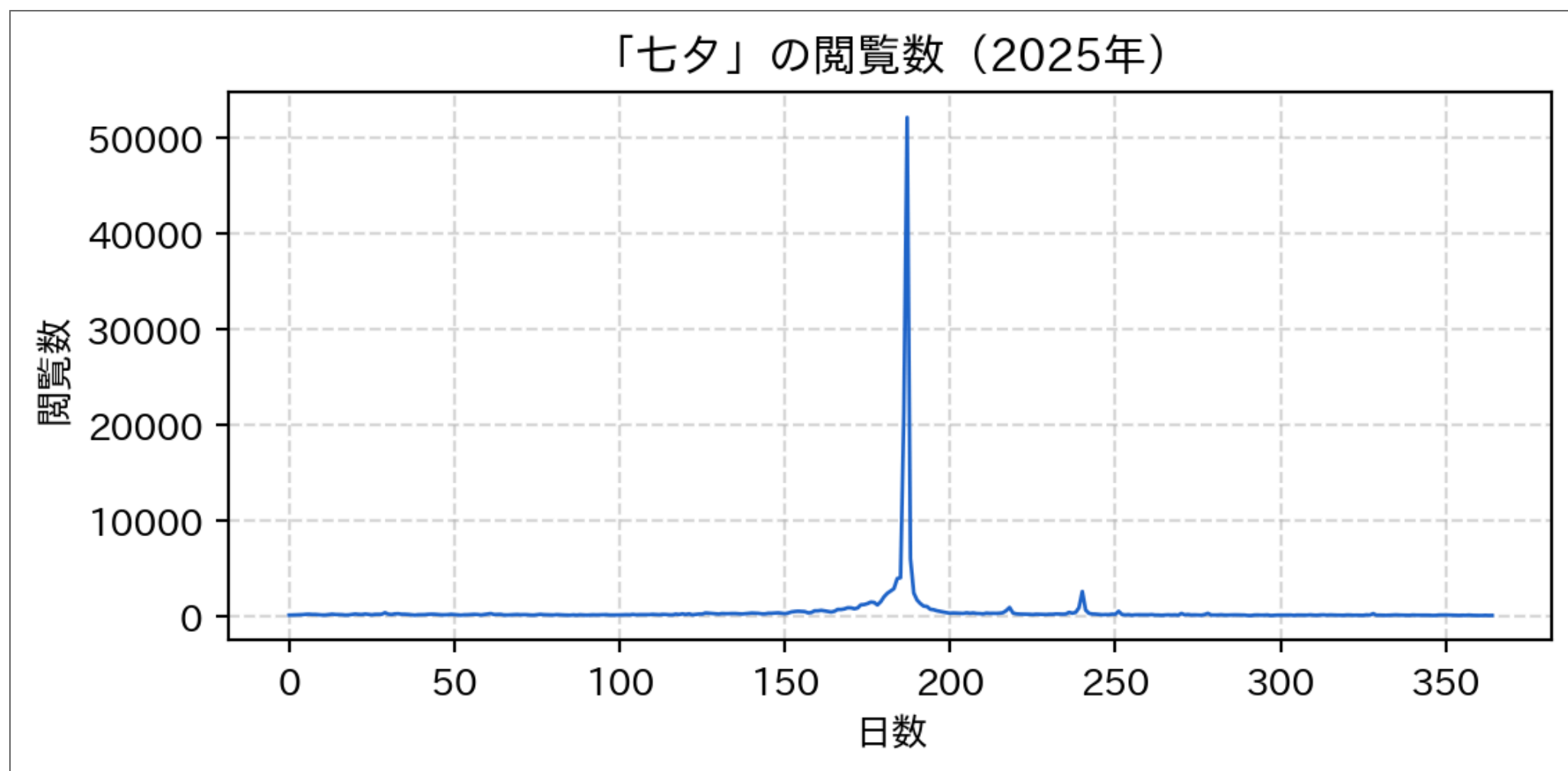
- 最大が中央値の約280倍。平均と中央値も大きくズレている
- グラフには 1日だけの大きな山 — では、いつ？ なぜ？

3



前回の最後に描いたグラフ

- 数値だけでは見落とすので **まず描く** — [データの可視化](#) の大原則
- APIから取得した **views** を、そのまま **plt.plot()** に渡した結果



▶ コードを見る

① 1日だけ突出している

- ほぼ平坦な中、**1日だけ大きな山（ピーク）**
 - **数値だけ見ていると気づけない**（アンスコムの教訓と同じ）
- そして見覚えがあるはず： [データの可視化の冒頭で見た 謎の図](#) は、実はこのデータ

ピークの「いつ」を特定する

- 山の **高さ** は `views.max()`。知りたいのは **「いつ」**
- `item["timestamp"]` も同じ内包表記で取り出すと、**views** と **同じインデックスで並ぶ** 日付の配列ができる
- `np.argmax()` は最大値の **インデックス** を返す（値ではない点に注意） — そのインデックスで **dates** を引く

Code

```
1 dates = np.array([item["timestamp"] for item in data["items"]])
2
3 max_i = np.argmax(views)
4 print(f"{dates[max_i]} に {views[max_i]} 回")
```

❗ ここで質問

記事は「七夕」。52,168回も読まれた**その1日**は、何月何日だと思う？

1 2025070700 に 52168 回

- **2025年7月7日 — 七夕の当日**
- 誰もが、**行事の当日にその行事を調べる**
 - 「七夕って何の日？」「短冊はなぜ？」 — 検索が記事に殺到する
- Wikipediaが読まれるタイミングが、API経由の数列ひとつに写っている

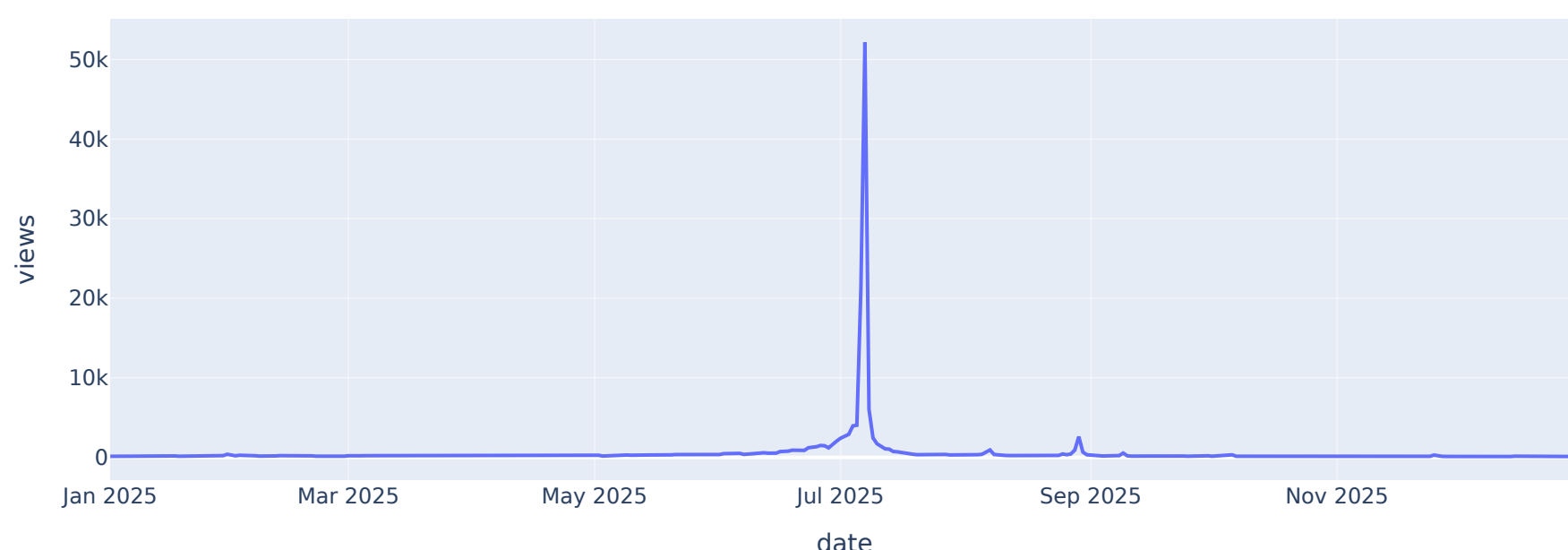
5

plotlyでも書いてみる

- pltだけではなく、plotlyでも可視化する
 - **Plotly入門** [🔗](#)
- `data["items"]` は、`pd.DataFrame()` に渡すと **そのままdfになる**

Code

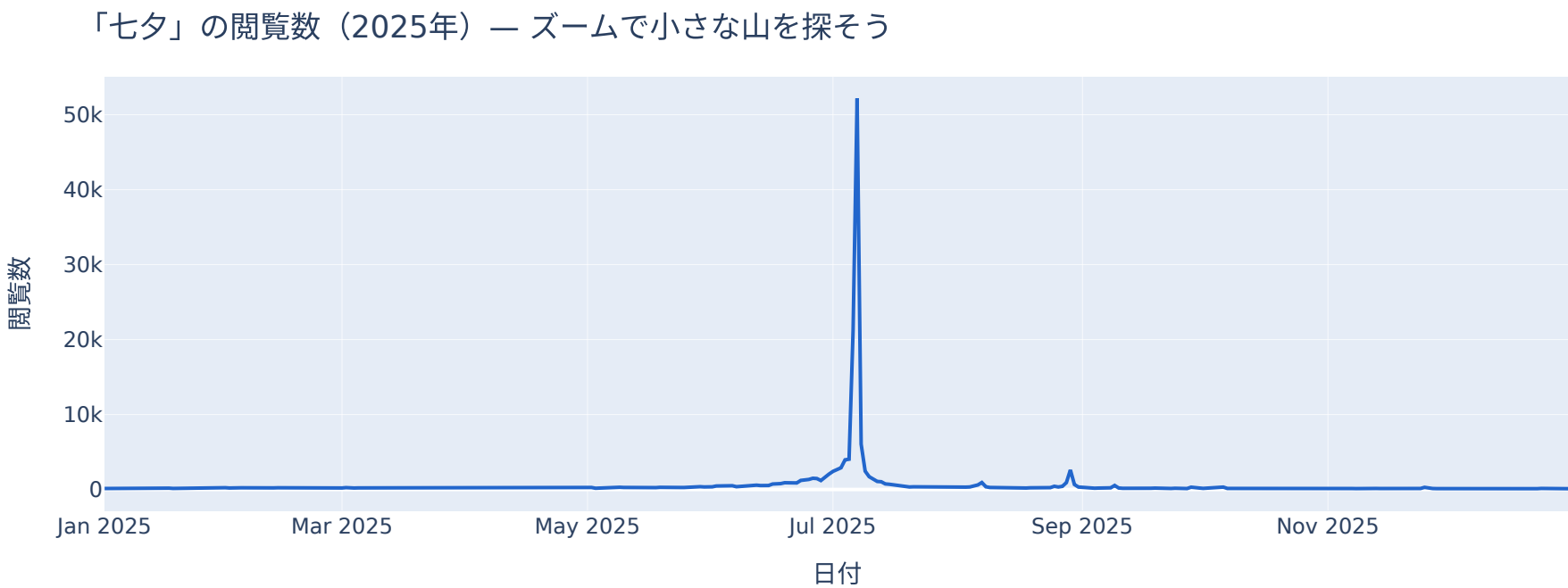
```
1 import pandas as pd
2 import plotly.express as px
3
4 df = pd.DataFrame(data["items"]) # dictのlistをdfへ
5 df["date"] = pd.to_datetime(df["timestamp"], format="%Y%m%d%H")
6
7 fig = px.line(df, x="date", y="views")
8 fig.show()
```



6

よく見ると、もう一つ山がある

- 7/7の山が大きすぎて、ほかの日は**足元で平ら**に見える
- 動くグラフなら確かめられる：**山の足元をドラッグで拡大**（ダブルクリックで全体に戻る）



- **8月29日にも小さな山**（2,603回） — 七夕でもないのに、なぜ？

7

8月29日の山の原因



- **伝統的七夕** — 旧暦7月7日にあたる日（2025年は8月29日）
 - 国立天文台が毎年発表している。7月7日は梅雨だが、**旧暦の七夕は夏空で天の川が見える**
- **ズームしないと見えない発見もある**
 - 軸の範囲は「正直さ」だけでなく「発見」の道具でもある

8

まとめ

やりたいこと	コード
まず描く	<code>plt.plot(views)</code>
最大値の位置	<code>max_i = np.argmax(views)</code>
位置から日付	<code>dates[max_i]</code> （同じindexの配列をリンク）
表にして動くグラフ	<code>pd.DataFrame(data["items"]) → px.line()</code>

- 統計量で「おかしい」と気づき、**図で正体を突き止める**
- 大原則 *Always plot your data first.* は **API由来のデータ** にもそのまま効く

9

演習A：1枚のグラフに2本の折れ線



「七夕」の2025年1年分を、**人間の閲覧**（**user**）に絞って **desktop** と **mobile-web** で取得し、**日付をx軸にして同じグラフに重ねて** 比較せよ。

0. URLの **all-agents** を **user** に変更
1. **access** を切り替えて、2つの応答を取得
2. それぞれ **pd.DataFrame(data["items"])** で表にし、**timestamp** から日付列を作成
3. **plt.plot(日付, views)** を2回呼び、凡例つきで重ね描き
4. グラフのタイトルに **max / min / mean** を表示

① URLの **all-agents** の意味

この部分は**誰の閲覧を数えるか**の指定。**all-agents** はbot・クローラー込み、**user** は**人間だけ**。「人がどの端末で読んだか」を聞いたら **user** が正しい。

⚠ 禁止事項

- 軸を揃えず2枚並べる → 比較しづらい
- 軸の範囲を片方だけ狭める → 印象操作（[matplotlib入門](#) 「軸の範囲で印象を操る」の罠）
- 2本を **配列の位置** で重ねる → このAPIは閲覧数0の日を応答から省くことがあり、2つの応答の日数と並びが一致する保証はない。**時系列は日付で対応づける**

▶ 解答例を見る

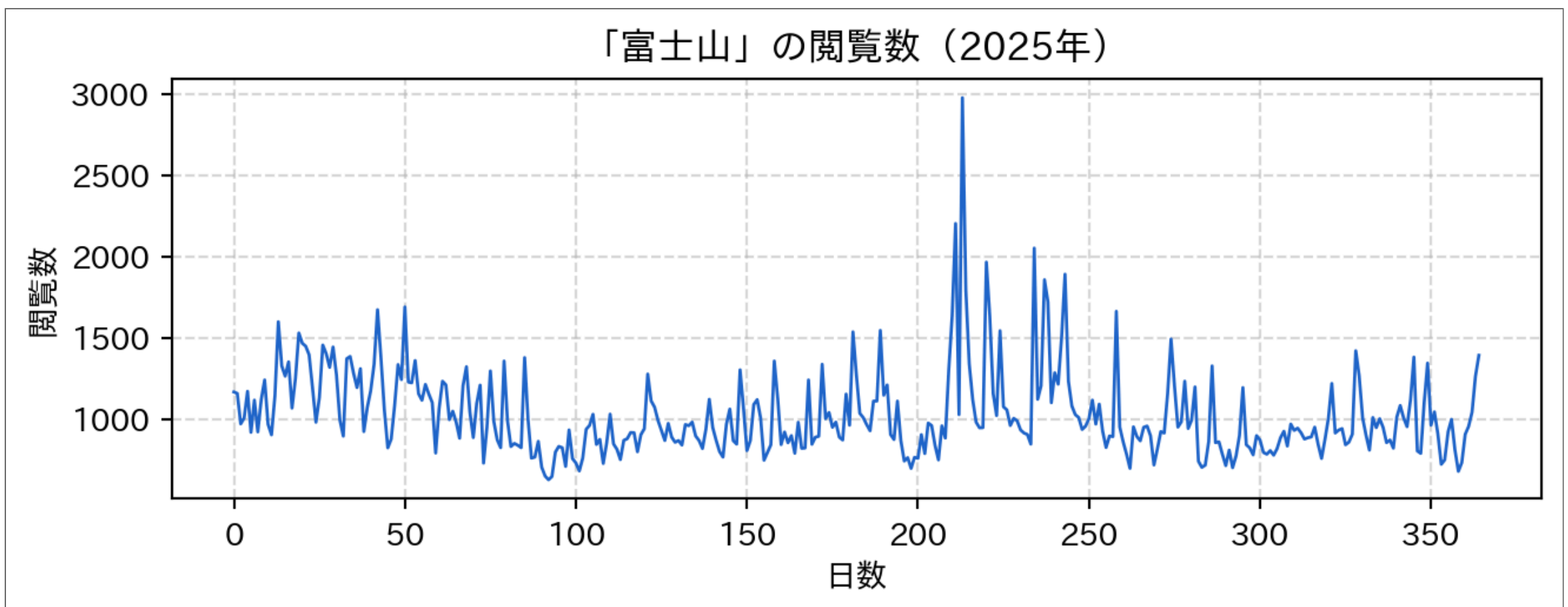
演習B：富士山には山があるか



「七夕」のコードの **記事名を 富士山 に変えるだけ** で2025年1年分を取得して描き、次の問いに答えよ。

- 問い：七夕のような **鋭い山（スパイク）** はあるか？
- 問い：スパイクがないなら、**何が見えるか**？ 自分の言葉で説明せよ

▶ 解答例を見る



i 針はない。でも「波」がある

- 七夕のような数百倍のスパイクは **ない**（最大でも中央値の約3倍）
- 代わりに **8月（登山シーズン）と1～2月** が高い、ゆるやかな波
 - 夏が高いのは分かる。冬はなぜ高い？
- **スパイクだけが発見ではない**。波（季節性）も、データが教えてくれる関心の動き

12

参考リンク

- [Wikimedia Analytics API](#)
- [Page metrics examples](#)
- [国立天文台「伝統的七夕」](#)

13